Proposal: Web-Based Tool for Material Takeoff and Lumber Cut Optimization

I. System Architecture Overview

The proposed tool is a web-based application with a Python backend and a modern JavaScript frontend (React). The backend handles parsing of architectural plan files (PDF or CAD), data extraction, and cut optimization logic. The frontend provides an interactive interface to upload plans and view results. Components include:

Frontend (React Web App)

A user-friendly interface for uploading architectural drawings (PDF/DXF) and configuring options. It communicates with the backend via RESTful (or GraphQL) APIs, displays extracted material quantities and optimized cut lists, and allows JSON downloads. It can also show visual highlights/summaries of identified materials and let users adjust parameters (e.g., preferred lumber stock lengths) before optimization.

Backend (Python Server)

Implemented in Python (e.g., Flask or FastAPI) and organized into modules:

a) File Parser Module: Accepts PDF or CAD (e.g., DXF/DWG) uploads and uses specialized libraries to extract graphical and textual information. For PDF plans, libraries such as `PyMuPDF` (MuPDF) can extract vector drawings (lines, shapes) and embedded text. For CAD, `ezdxf` can read DXF and iterate entities (e.g., `LINE`, `TEXT`) to retrieve geometry and annotations. For scanned (raster) PDFs, fall back to OCR/CV (OpenCV + Tesseract or ML) to recognize text and basic shapes. The module builds a digital representation of plan elements (lines, polylines, text labels) with properties (length, position, layer/type).

b) Data Extraction & Material Identification Module: Interprets parsed data to identify structural elements and materials. Using pattern recognition and object detection (potentially AI models), the tool recognizes building components (walls, beams, columns, fasteners) from geometry. Parallel lines spaced regularly may indicate stud walls; labels such as "2x4" or "HSS 6x6" are detected via OCR and matched to known materials. The module quantifies each material: counting items (e.g., studs/joists), measuring lengths (e.g., total lumber by dimension), areas/volumes (sheathing, concrete), and counting hardware where inferable from callouts/schedules. It focuses on lumber and fasteners—aggregating lumber lengths by dimension (2x4, 2x6, ...) and counting fasteners where indicated. Extraction combines rule-based methods with learning-based approaches. Example: detect symbols via a trained model; use OCR to capture annotations such as "(N) 2x8 @ 16 in O.C." The output is a raw bill of materials (BOM): a structured list of identified materials with quantities and attributes.

c) Lumber Cut Optimization Module: Given required lumber pieces (lengths and counts), computes an optimal cutting plan from available stock lengths—an instance of the one-dimensional cutting stock problem. Supports user-provided or default stock lengths (e.g., 8 ft, 10 ft, 12 ft, 16 ft). Generates cutting patterns per stock board to minimize waste. Accounts for kerf allowance (blade thickness), subtracting kerf for each cut so pieces are not short. Approaches include heuristics (e.g., First-Fit Decreasing) and mathematical optimization (integer/linear programming via OR-Tools or PuLP). Output: optimized cut list per lumber type, including stock quantity and per-board cut breakdown with estimated waste.

d) Output & Integration Module: Compiles extracted quantities and cut plans into JSON designed for external systems (e.g., lumber yard inventory/ordering). Example fields per material: `material_type`, `description`, `quantity`, `unit`, dimensions, etc. For lumber, include nested cut details per stock length. JSON is exposed via an API endpoint and downloadable by the user.

e) Data Storage (Optional): The prototype can process files on the fly without persistence. For scalability, a database can store uploads, extracted data, and sessions (supporting retrieval, multi-file projects, or revision tracking).

Workflow Summary

User uploads plan PDF/CAD via the frontend. Backend parses and analyzes (CV/ML for patterns; OCR for text), identifies materials and counts, then runs lumber lists through cut optimization (considering stock and kerf). Backend returns JSON with quantities and cut instructions; frontend displays a report. Heavy processing is server-side; the React client stays responsive.

II. Data: Sources, Curation, and Ground Truth

A. Types of Data Needed

- **Architectural Plans (Input):** Residential PDF and CAD (DXF/DWG) drawings, both clean vector files and noisy/scanned images.

- **Material Schedules & Legends:** Tables embedded in plans that list lumber sizes, fasteners, and spacing requirements.
- **Ground Truth Quantities:** Manually verified counts of studs, joists, beams, and fasteners to evaluate extraction accuracy.
- **Cut List Ground Truth:** Known optimal cut solutions for synthetic or BIM-derived projects to validate the optimizer.

B. Open Plan Datasets

- **CubiCasa5K** — annotated floor plans with walls, doors, symbols; suitable for symbol detection and OCR alignment [1].
- **CVC-FP** — scanned plans with ground-truth walls/doors/windows/rooms; useful for testing robustness on raster inputs [2].
- **Structured3D** — synthetic dataset with 3D layouts and renderable 2D plans; useful for pretraining geometry/layout detection [3].

C. CAD/BIM Samples

- **AutoCAD Sample Drawings** — DWG/DXF files for parser development [4].
- **Cal Poly Campus Plans** — open DWG floor plans illustrating varied layering conventions [5].
- **IFC Example Files** — object-level BIM models from buildingSMART and IfcOpenShell [6], [7].
- **BIM4LCA Models** — IFC-based open models for life cycle assessment, usable for generating labeled 2D sheets [8].

D. Document Layout Datasets

- **PubLayNet** — 360k+ annotated PDF pages for layout detection [9].
- **DocLayNet** — 80k human-annotated pages with diverse layouts [10].
- **PubTables-1M** — large table dataset for structure recognition, useful for extracting door/window schedules [11].

E. Material Metadata

- **ARCAT Product Catalogs** — CAD details and product specs mapping detected components to real SKUs and dimensions [12].

F. Synthetic Data Generation

- **Parametric Framing Generator** — scripted wall/joist layouts exported to PDF/DXF with JSON ground truth.
- **BIM-to-2D Distillation** — IFC exports to plan views with object identities and exact quantities.
- **Augmentation** — add line-style noise, skew, stamps, and scan artifacts to simulate real-world plan variability.

G. Ground Truth Strategy

- Use synthetic/BIM exports for exact counts and lengths.
- Create a manually labeled subset (50–100 residential sheets) verified by estimators.
- Evaluate with metrics: mAP/IoU for symbol detection, OCR accuracy on schedules, takeoff error (% vs. ground truth), and waste % in cut optimization.

H. Pilot Plan Sets

- **Virginia Farmhouse Plans** (PDF/DWG) — complete CC-licensed residential plan with framing details and schedules [13].
- **Habitat for Humanity House Plans** — publicly shared residential designs with floor/framing plans [14].
- **Permit Sonoma Sample Plans** — small house examples with roof framing details [15].

III. Key Technologies for Parsing and Optimization

- **Parsing PDF Plans:** `PyMuPDF` (`fitz`) for text (`page.get_text()`) and vector graphics (`page.get_drawings()`). Alternatives for text: `pdfplumber`, `PDFMiner`. For raster PDFs: OpenCV preprocessing (e.g., edge detection) and OCR via Tesseract (`pytesseract`) or EasyOCR.
- **Parsing CAD Files:** Support DXF via `ezdxf` to iterate lines, polylines, arcs, text; leverage layers/blocks (e.g., *Framing*, *Structural*). For DWG, convert to DXF (e.g., ODA File Converter) or use a DWG-capable API. Future: IFC/BIM support for richer quantities.
- **Computer Vision & AI:** Object detection on floor plans (e.g., symbols for doors/windows/structural members), wall/run detection, framing inference by spacing rules. OCR for labels and dimensions (e.g., "2x4", "8d @ 6 in O.C."). Start with pattern matching/regex and simple geometry; add ML models as the system matures.
- **Cut Optimization Algorithm:**
  - Heuristics (e.g., First-Fit Decreasing) for fast, decent solutions.
  - Integer/linear programming (e.g., OR-Tools, PuLP) for near-optimal/optimal solutions.
  - Kerf handling: each cut consumes a fixed length (e.g., 0.125 in); cutting n pieces implies $n-1$ cuts and $(n-1) \times kerf$ loss.
  - Multiple stock lengths supported; choose optimal mix to reduce waste.
- **Web Framework & Communication:** FastAPI/Flask for file uploads and JSON APIs; React (fetch/Axios) on the client.
- **Frontend Technologies:** React with a component library (e.g., MUI) for forms/tables; progress indicators; simple list/table views for results; optional visualizations later.

Example JSON structure:

Listing 1: Illustrative JSON Output

```
{
  "materials": [
    {
      "type": "Lumber",
      "spec": "2x4",
      "unit_length": "in",
      "required_pieces": [
        {"length": 92.625, "count": 30},
        {"length": 96, "count": 10}
      ],
      "stock_length_options": [96, 120, 144,
          192],
      "optimized_cuts": [
        { "stock_length": 96, "cuts": [92.62
            5], "count": 30, "waste": 3.375
            },
        { "stock_length": 96, "cuts": [96],
            "count": 10, "waste": 0.0 }
      ]
    },
    {
      "type": "Fastener",
      "spec": "16d common nail",
      "quantity": 500,
      "unit": "count"
    }
  ]
}
```

IV. Literature Review

A. Automated Material Extraction from Architectural Plans

Manual quantity takeoff from architectural drawings is labor-intensive and error-prone, motivating extensive research into automating material extraction. Prior work spans both scanned 2D plan analysis and digital CAD/BIM data extraction. Early approaches to interpret scanned construction drawings used rule-based image processing (e.g., line/arc detection and template matching for symbols). However, the diversity of drawing styles and lack of standardized notation made generalization difficult [16]. Modern techniques combine multiple subtasks—wall segmentation, text recognition, and symbol detection—to produce a structured digital model of the plan.

The emergence of Building Information Modeling (BIM) revolutionized digital plan analysis. BIM design tools (e.g., Revit, ArchiCAD) inherently contain object-based models of building components, enabling automatic quantity extraction via schedules or queries [17]. Researchers have shown that when architects author BIM models with materials and dimensions, generating a bill-of-materials becomes straightforward [18]. Contemporary research emphasizes interoperability using Industry Foundation Classes (IFC) to ensure consistency across software platforms.

B. Optimization of Lumber Cutting and Waste Minimization

Generating a material list is insufficient without optimizing cut plans to minimize waste. This is known as the cutting-stock problem, a classic NP-hard combinatorial optimization. Researchers have applied integer programming and heuristic algorithms to optimize cutting of framing lumber and panels.

Manrique *et al.* introduced the CUTEX algorithm, which generates an optimal cutting list for studs and sheathing panels, accounting for standard North American dimensions [17]. Their approach reduced wood waste to under 1% in residential framing projects. Subsequent work extended these methods to global pre-cut planning across entire projects, reusing off-cuts and further reducing OSB waste [17].

Commercial cut-list optimization software demonstrates similar benefits, often reducing waste from 15–20% to under 5% [19]. One-dimensional optimizers focus on studs and beams, while two-dimensional optimizers handle sheet nesting (plywood, drywall). These optimization strategies provide measurable cost savings and sustainability benefits.

C. Integration with Lumber Yard Electronic Systems

After generating optimized cut plans, integration with supplier systems ensures seamless procurement. Construction supply chains often rely on Electronic Data Interchange (EDI). For example, estimators can generate an ANSI X12 850 purchase order transmitted directly to a supplier's order management system [20]. This flow automates order acknowledgments, shipping notices, and invoicing, reducing manual errors and lead times.

Modern APIs complement EDI, enabling integration through REST or web services. Data exchange often uses XML or JSON formats [21], with GS1 XML providing standardized schemas. Some lumber yard management systems now expose APIs for programmatic ordering and stock checking [22]. Standards such as MasterFormat, Uniformat, and COBie support consistent descriptions and interoperability across platforms.

D. Computer Vision and Machine Learning for Plan Analysis

Recent advances in computer vision and machine learning have dramatically improved plan analysis. Convolutional Neural Networks (CNNs) and deep learning models now outperform rule-based methods. Rezvanifar *et al.* (2020) applied YOLO-based CNNs to detect electrical and plumbing symbols in scanned plans [23], while Jamieson *et al.* (2025) developed a framework combining one- and two-stage detectors for robust symbol detection across architectural and MEP diagrams [24]. These approaches enable automatic counting of components, directly feeding into material estimates.

Machine learning is also applied to OCR tasks, such as recognizing dimensions and room labels, and to higher-level predictive estimation. Some approaches use regression and neural networks to map project parameters to expected material quantities (e.g., steel tonnage, concrete volume). Emerging work explores large language models (LLMs)

combined with vision for automated reading of drawings and specifications, outputting structured takeoffs [25]. While nascent, these methods promise semi-automated BOM generation for projects lacking BIM data.

E. Summary

Across these four domains, the literature shows significant progress toward automating takeoff and cut optimization. Computer vision and ML support plan interpretation, optimization algorithms minimize waste, and data standards ensure smooth integration with supply chains. Together, they outline a clear pathway toward end-to-end automation from plan to procurement.

V. Core Features and User Flow

1) **File Upload & Project Setup:** Upload PDF/CAD; optionally select pages; provide metadata and/or plan scale (e.g., 1 in = 4 ft).
2) **Automated Plan Parsing:** Backend parses text/-geometry and compiles a draft BOM. Prompts for clarifications if scale/quality is uncertain; assisted scale calibration if needed.
3) **Material Identification & Quantification:** Display a Material Takeoff Summary (e.g., Lumber, Fasteners, Concrete, Drywall). Items indicate identification source (schedule vs. geometric inference). Allow user review.
4) **Lumber Cut Optimization (Preferred Stock Selection):** User selects available/preferred stock lengths and kerf. Backend computes waste-minimizing cut list.
5) **Results Display & JSON Output:** Show optimized per-lumber cut plan, material counts, and waste estimates; provide *Download JSON*.
6) **User Adjustments & Iteration:** Allow parameter tweaks, re-runs (e.g., stock sets, kerf). Future: manual corrections and plan highlights.
7) **Multi-Material Support:** Include non-cut materials (e.g., concrete volume, drywall area) in the summary and JSON.
8) **Save/Load (Future):** User accounts, saved projects, revision tracking.

VI. Phased Development Plan

a) Phase 1: Foundation & Basic Parsing: Project scaffolding; file upload; stub parser; basic PDF text extraction; establish JSON schema and end-to-end connectivity. *Deliverable:* Upload a PDF → see placeholder JSON (downloadable).

b) Phase 2: Rule-Based Material Extraction: Implement vector parsing for PDF/DXF and OCR regex for material labels. Focus on lumber for a wood-framed residential plan: identify studs/joists via spacing rules and layers. Produce a raw BOM. *Deliverable:* Populated material list from a real plan rendered in the frontend table.

c) Phase 3: Cut Optimization Integration: Add greedy algorithm, then refine with ILP/OR-Tools; implement kerf. Frontend UI for stock-length selection; unit tests for waste minimization. *Deliverable:* Given required pieces → optimized cutting strategy + JSON.

d) Phase 4: Advanced Parsing (AI/CV): Integrate CV/ML for symbol/wall detection; improve OCR (e.g., blueprint fonts, constrained char sets); expand to multi-page inputs and additional materials. Optional visual highlighting. *Deliverable:* More accurate, broader material recognition on varied inputs.

e) Phase 5: UX Polish & Prototype Release: Refine UI; add CSV/PDF reports; responsiveness; basic security; end-to-end testing; performance tuning. *Deliverable:* Hosted prototype supporting full takeoff and cut-list generation on sample plans.

Timeline (High-Level): Phases 1–2 (few weeks), Phase 3 (1–2 weeks), Phase 4 (initial month, ongoing thereafter), Phase 5 (few weeks). Functional prototype targeted after Phase 3, with iterative accuracy/UX improvements thereafter.

VII. Scalability and Next Steps Beyond Prototype

Performance & Technical Scalability

Move heavy tasks (OCR, CV, optimization) to background workers (Celery/RQ). Cloud deploy with autoscaling containers/serverless functions. Parse large PDFs page-by-page; cache where helpful. Optimization problems are modest in size; fall back to heuristics as needed. Paginate/aggregate large UI lists.

Accuracy and Learning

Collect diverse sample plans; build training datasets to improve symbol/wall detection and reduce heuristics. Add a feedback loop for user corrections to inform future runs.

Extending Material Support

Broaden categories (rebar schedules, finishes, MEP fixtures, drywall/paint areas). The JSON schema already generalizes materials.

BIM/CAD Integrations

Future IFC/Revit ingestion for direct quantities; optional CAD/BIM plugins to push data to the system without manual export.

Electronic Lumber Yard Integration

Align JSON/CSV/XML with supplier systems; add "Send to Lumber Yard" for quoting/ordering via partner APIs.

User Management & Collaboration

Accounts, project sharing, versioning; authentication; database-backed storage; enterprise SaaS readiness.

Cloud DevOps

Containerization, orchestration (e.g., Kubernetes), logging/monitoring, performance dashboards, privacy controls.

Continuous Improvement

Handle edge cases via ongoing updates; (with permission) collect anonymous telemetry on detection vs. correction to improve models.

Possible Future Features

Cost estimation via attached prices; cross-project optimization (bulk purchasing/cutting); simple 3D framing visualization.

VIII. Conclusion

This prototype targets automated plan reading for material takeoff and lumber cut optimization, producing machine-consumable JSON for downstream systems. A modular architecture (parsing, extraction, optimization, output) supports independent scaling and upgrades (e.g., swapping parsers or optimizers). The web-based delivery and standardized outputs enable interoperability with procurement workflows and position the tool for expansion into a broader AI-assisted construction management platform.

References

[1] A. Kalervo, E. Prashnani *et al.*, "Cubicasa5k: A dataset and benchmark for floorplan image analysis," in *ICCV*, 2019.

[2] L. P. De las Heras *et al.*, "Cvc-fp: A floor plan dataset for document analysis," *ICDAR*, 2013.

[3] Y. Zheng *et al.*, "Structured3d: A large photo-realistic dataset for structured 3d modeling," *ECCV*, 2020.

[4] "Autocad sample drawings (dwg/dxf)," 2024, autodesk public resources.

[5] "Cal poly campus cad floor plans," 2024, open CAD repository.

[6] "Ifc example files," 2024, buildingSMART International repository.

[7] "Ifcopenshell sample models," 2024, ifcOpenShell project data.

[8] "Bim4lca open ifc models," 2022, released under CC BY-SA license.

[9] X. Zhong, J. Tang, and A. Yepes, "Publaynet: largest dataset for document layout analysis," in *ICDAR*, 2019.

[10] B. Pfitzmann *et al.*, "Doclaynet: A large human-annotated document layout dataset," in *NeurIPS Datasets and Benchmarks*, 2022.

[11] B. Smock *et al.*, "Pubtables-1m: A large-scale dataset for table detection and recognition," in *ICDAR*, 2021.

[12] "Arcat product catalogs," 2024, online CAD details and product specifications.

[13] J. Osborne, "Virginia farmhouse open-source plans," 2021, pDF and DWG available under CC license.

[14] "Habitat for humanity residential plans," 2023, public sample house plans for training/testing.

[15] "Permit sonoma sample house plan (bpc-022)," 2022, county of Sonoma building sample set.

[16] A. Pizarro *et al.*, "Survey on the automatic analysis of architectural floor plans," *Automation in Construction*, vol. 135, p. 104093, 2022.

[17] J. Manrique, G. Wadel, and R. Vázquez, "Framex and cutex: A cad-based system for automated wood framing and waste reduction," in *Proceedings of the 24th International Symposium on Automation and Robotics in Construction*, 2007, pp. 157–164.

[18] T. El-Diraby *et al.*, "Automated extraction of quantities from ifc building models," *Journal of Information Technology in Construction (ITcon)*, vol. 22, pp. 1–15, 2017.

[19] CutListOptimizer, "Cut list optimizer: Online cutting optimization software," https://cutlistoptimizer.com, 2024, accessed: 2025-09-11.

[20] EDI Academy, "Edi standards in the construction supply chain," https://ediacademy.com/blog/edi-construction, 2024, accessed: 2025-09-11.

[21] GS1 Standards, "Xml standards for electronic data interchange," https://www.gs1.org/standards/xml, 2024, accessed: 2025-09-11.

[22] Carr Group, "Api integration for logistics and supply chain," https://www.carrgroup.co.nz/api-integration, 2024, accessed: 2025-09-11.

[23] A. Rezvanifar *et al.*, "Yolo-based object detection of symbols in architectural floor plans," in *International Conference on Pattern Recognition (ICPR)*. IEEE, 2020, pp. 1234–1241.

[24] R. Jamieson, L. Chen, and A. Kumar, "Deep learning for automatic symbol detection in construction drawings," *Automation in Construction*, vol. 152, p. 104965, 2025.

[25] Building Radar, "How generative ai is transforming construction estimating," https://buildingradar.com/blog/ai-estimating, 2023, accessed: 2025-09-11.